Maze Solver

Languages allowed: Java **or** Python only.

Critical rule. You may **not** use any built-in data structure or algorithm libraries for the **core maze/graph/queue behavior**.

- **Java – allowed:**
  - Primitive arrays (`int[]` , `char[]` , etc.), `String` , `StringBuilder` , I/O, exceptions, math utilities
  - `HashMap` for auxiliary *lookup* only (e.g., map maze names to grids, map IDs to indices)
- **Python – allowed:**
  - `dict` for auxiliary *lookup* only
  - `list` **only as a fixed-size raw array buffer** inside your own structures (you must implement your own Vector/Queue wrappers; you may not expose Python lists directly as the main data structure)

Not allowed (core structures must be your own):

- Java: `ArrayList` , `LinkedList` , `Vector` , `Stack` , `Queue` , `Deque` , `PriorityQueue` , `HashSet` , `TreeMap` , etc.
- Python: built-in `list` as a dynamic container, `set` , `heapq` , `collections.deque` , `queue.Queue` , or any graph/algorithm libraries.

You must implement from scratch:

- A **grid-based graph representation** of the maze (implicit or adjacency list)
- A **FIFO Queue** to perform **Breadth-First Search (BFS)**
- Any **dynamic array / Vector** style backing you need

> This project uses **BFS only** (no DFS, no heap sort, no priority queues).

Goal

Build a **command-line maze solver** that:

- Reads a maze from a separate text file
- Represents the maze as a **grid graph** (each open cell is a vertex; edges connect neighboring open cells up/down/left/right)
- Uses **Breadth-First Search (BFS)** to find the **shortest path in number of moves** from a unique start cell `S` to a unique goal cell `G`
- Outputs both:
  - Whether the maze is solvable and the **shortest path length**
  - The actual **path** as a sequence of grid coordinates

The solver is driven entirely by commands read from a **commands file** (no interactive input).

Input & Command-Line Interface (CLI)

Invocation

Your program must accept a single argument: the path to a plain-text **commands file**.

- Java:

  ```
  java Main commands.txt
  ```

- Python:

  ```
  python3 main.py commands.txt
  ```

Behavior when missing/invalid

If the commands file path is missing **or** the file cannot be opened:

```
USAGE: <program> <commands_file>
```

…and exit with a **non-zero** status code. Do **not** crash with a stack trace.

Parsing rules (commands file)

- Each **non-empty** line is a command to execute.
- Lines beginning with `#` are **comments** and must be ignored.

- Commands are **case-insensitive**; arguments are **case-sensitive** unless stated otherwise.
- Arguments are separated by spaces. Text wrapped in double quotes (`"like this"`) must be treated as a single token (without the quotes).
- Extra whitespace at the beginning or end of lines must be ignored.
- All output must go to **stdout** and follow the exact formatting rules in this spec.

Maze file format

The maze itself is stored in a **separate text file**, whose path is given to the `LOAD_MAZE` command.

Maze file format:

1. First line: two integers `ROWS COLUMNS` (both > 0)
2. Next `ROWS` lines: each line contains exactly `COLUMNS` characters, each from the set:
   - `#` – wall / blocked cell (not walkable)
   - `.` – open cell (walkable)
   - `S` – start cell (exactly one in the maze)
   - `G` – goal cell (exactly one in the maze)

Example maze file `maze1.txt` :

```
5 7
#######
#S...G#
#.###.#
#.....#
#######
```

Coordinates use **0-based** indexing:

- Row indices: `0..ROWS-1` , top to bottom
- Column indices: `0..COLUMNS-1` , left to right

Functional Requirements

We model a **single current maze** at a time. BFS is always performed on the *currently loaded* maze.

Commands are case-insensitive; arguments are space-separated.

1. `LOAD_MAZE path`

Load a maze from the given file.

- If another maze is already loaded, it is **discarded** and replaced.
- You must:
  - Open the file at `path`
  - Read `ROWS` and `COLUMNS`
  - Read the next `ROWS` lines of the grid
  - Validate:
    - Exactly **one** `S`
    - Exactly **one** `G`
    - Every line has exactly `COLUMNS` characters

On **success**, print:

```
MAZE_LOADED rows=R cols=C
```

where `R` and `C` are the parsed dimensions.

On failure (file not found, wrong dimensions, missing `S` / `G` , etc.), print:

```
ERROR: INVALID_MAZE
```

…and clear any partially loaded maze (i.e., no maze should be considered loaded).

2. `PRINT_MAZE`

Print the currently loaded maze **exactly** as stored.

Format:

```
MAZE
#######
#S...G#
#.###.#
#.....#
#######
```

- First line: `MAZE`
- Then `ROWS` lines of the maze characters
- If no maze is currently loaded, print:

```
ERROR: NO_MAZE
```

3. `SOLVE`

Run **Breadth-First Search (BFS)** from the start `S` to the goal `G` on the current maze.

- If **no maze** is loaded, print:

  ```
  ERROR: NO_MAZE
  ```

- Otherwise:
  - Construct the implicit graph:
    - Each walkable cell `{'.', 'S', 'G'}` is a vertex
    - Edges connect orthogonally adjacent walkable cells (up, down, left, right)
  - Run **BFS** from `S` until reaching `G` or exhausting all reachable cells.
  - If **no path** exists:
    - Internally clear any previous path (no valid solution)
    - Print:

      ```
      UNSOLVABLE
      ```

  - If a path **does** exist:
    - Recover the **shortest path in terms of number of moves** (edges) using parent pointers.
    - Store the path internally (for `PRINT_PATH`).
    - Print:

      ```
      SOLVED
      -> length=L
      ```

      where `L` is the number of **moves** (edges) from `S` to `G`.

> Note: If the path uses `k` cells including `S` and `G`, then `length = k - 1` (since length is counted in **steps**, not cells).

4. `PRINT_PATH`

Print the most recently computed BFS path from `S` to `G`.

- If no maze is loaded:

  ```
  ERROR: NO_MAZE
  ```

- Else if `SOLVE` has not yet been successfully run, or the last `SOLVE` reported `UNSOLVABLE`:

  ```
  PATH
  NO_PATH
  ```

- Else (there is a valid stored path), print:

```
PATH
-> (r0,c0)
-> (r1,c1)
...
-> (rk,ck)
```

Where:

- `(r0,c0)` is the **start** cell `S`
- `(rk,ck)` is the **goal** cell `G`
- Coordinates use **0-based** `row,column` pairs

Example:

```
PATH
-> (1,1)
-> (1,2)
-> (1,3)
-> (1,4)
-> (1,5)
```

5. `CLEAR`

Clear the current maze and any stored path. After this:

- There is **no loaded maze**.
- `PRINT_MAZE`, `SOLVE`, and `PRINT_PATH` should behave as if no maze exists.

On success, you may either print nothing or:

```
CLEARED
```

(Choose one policy and keep it consistent; document it in your README.)

6. `QUIT`

Terminate processing immediately. Any lines after `QUIT` are ignored. No special output is required here beyond previous commands.

Output Format (Summary)

Your outputs must follow these conventions:

- Headers on their own line: `MAZE`, `SOLVED`, `UNSOLVABLE`, `PATH`, etc.
- Path lines start with `->` followed by either:
  - `length=L`
  - or coordinate pairs `(r,c)`
- All error messages start with `ERROR:` (all caps, followed by a brief error code or description).

Exact spelling, capitalization, and spacing matter for automated grading.

Data Structure Constraints

You must implement your own **maze, graph, and BFS queue** representations.

Maze and graph representation

- Internally, you must store the maze in some grid form:
  - Java: typically `char[][]` or equivalent
  - Python: your own matrix structure built on top of fixed-size lists
- The **graph** can be:
  - **Implicit**: neighbors of a cell `(r,c)` computed on the fly by checking `up/down/left/right`
  - Or a manually maintained **adjacency list** using your own "Vector" or linked-list nodes per cell

You may **not** use built-in dynamic containers (`ArrayList` , Python `list` as dynamic list, etc.) for the **graph structure** itself.

BFS queue

You must implement your own **FIFO Queue** for BFS:

- Implemented using:
  - A circular array, **or**
  - A manually implemented linked list

Required operations:

- `enqueue(x)` – add to tail
- `dequeue()` – remove from head
- `isEmpty() / size()`

You may **not** use any built-in queue or deque.

Auxiliary structures

- **Visited and parent tracking** for BFS may use:
  - Arrays indexed by node index (recommended), or
  - Java `HashMap` / Python `dict` keyed by a cell index or coordinate
- If you need a dynamic array (for example, to store the final path), you must implement your own **Vector-like** structure on top of a fixed-size array buffer (e.g., double capacity when full).

Example Session

Example maze file: `maze1.txt`

```
5 7
#######
#S...G#
#.###.#
#.....#
#######
```

Here:

- `S` is at `(1,1)`
- `G` is at `(1,5)`
- One shortest path (in moves) is:
  `(1,1) -> (1,2) -> (1,3) -> (1,4) -> (1,5)`,
  which has `length = 4` moves.

Example commands file: `commands.txt`

```
# Load and solve a simple maze
LOAD_MAZE maze1.txt
PRINT_MAZE
SOLVE
PRINT_PATH
CLEAR
PRINT_MAZE
QUIT
```

Possible output

```
MAZE_LOADED rows=5 cols=7
MAZE
#######
#S...G#
#.###.#
#.....#
#######
SOLVED
-> length=4
PATH
-> (1,1)
-> (1,2)
-> (1,3)
-> (1,4)
-> (1,5)
CLEARED
ERROR: NO_MAZE
```

Explanation (informal):

- `LOAD_MAZE maze1.txt` loads the 5×7 maze successfully.
- `PRINT_MAZE` prints it exactly.
- `SOLVE` runs BFS and finds a shortest path of 4 moves from `S` to `G`.
- `PRINT_PATH` outputs the sequence of coordinates along that path.
- `CLEAR` removes the maze and the stored path.
- `PRINT_MAZE` now fails with `ERROR: NO_MAZE` because nothing is loaded.

Performance Expectations

Use BFS and basic array-based structures to meet the following complexity targets:

- Building any adjacency representation (if you choose explicit adjacency lists):
  O(R × C)
- `SOLVE` (BFS on grid):
  - Time: **O(V + E)** where `V ≈ R × C`, `E ≤ 4V`
    So effectively `O(R × C)`

    - Space: **O(R × C)** for visited/parent/path structures
- `PRINT_MAZE`:
    - Time: **O(R × C)** (printing all cells)
- `PRINT_PATH`:
    - Time: **O(L)** where `L` is the number of cells on the path
    - Space: if reconstructing each time, **O(L)**; if reusing stored path, printing is **O(L)** with no extra allocations

These bounds are easily satisfied if you implement BFS and your queue correctly.

Deliverables

1. **Source code** in Java or Python that:
    - Reads and processes the commands file
    - Reads and validates maze files
    - Implements BFS with your own queue and data structures
2. A short **README** (≤ 1 page) that explains:
    - Your maze representation and BFS design
    - Your queue implementation and any helper structures
    - How to compile and run your program (with example commands)
3. A **test transcript**:
    - At least one commands file
    - All required maze files
    - The corresponding program output, demonstrating:
        - Valid solvable maze
        - Valid unsolvable maze
        - Missing/invalid maze file
        - Error handling for commands

Academic Integrity & Restrictions

All submitted code and written work must be **your own**. Sharing, copying, or submitting work created by another individual (including code or text taken from the internet or AI tools without authorization)

will be treated as academic dishonesty and may result in significant penalties, up to and including failure in the course or dismissal from the university.

- **Disallowed:**
  - Reusing or lightly modifying solutions found online (GitHub, forums, etc.)
  - Using library implementations of queues, graphs, BFS, or pathfinding
  - Using any disallowed data structures for the core behavior
- **Allowed:**
  - Your own implementations of queues, vectors, graph representations, and BFS
  - Basic language features: I/O, primitives, strings, exceptions, and simple math/utilities

Refer to the course syllabus and university catalog for the full academic integrity policy.